

자료구조 실습 보고서

[제 07 주] 스택 – 기본 기능

제출일 : 2018 년 4 월 24 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘, 자료구조

선입후출을 특징으로 갖는 Stack 구조를 구현하였다. 맨 처음에 넣은 원소의 위치가 Bottom 이 되고, 맨 나중에 넣은 원소의 위치가 Top 이 된다.

원소를 추가하거나 제거할 때에는 Top 의 위치가 변동하게 되며 Bottom 의 위치는 변하지 않는다.

Stack 의 특성을 구현하는 함수들은 인터페이스로 미리 정의해놓고 Model 에서 상속받는 식으로 구현하였다.

```
public boolean push(T anElement)
```

공간이 남았을 시, top 의 위치를 현재 위치에서 한 칸 올리고 확보된 위치에 주어진 anElement 를 넣는다.

공간이 없어 실패하였을 시 false 를 반환하고, 그 이외에는 true 를 반환한다.

```
public T pop()
```

현재 top 의 위치에 있는 원소를 제거하고, 제거된 원소를 반환한다.

현재 공간이 비어 제거할 원소가 없을 시 null 을 반환한다.

b. 함수 설명서

■ AppController

- public void run() : 프로그램 실행

■ AppView

- public int inputInt() : 정수를 키보드로부터 입력받아 반환한다.
- public String inputString() : 문자열을 키보드로부터 입력받아 반환한다.
- public char inputCharacter() : 메시지 출력 후, 문자열을 키보드로부터 입력받고 가장 첫 문자를 반환한다.
- public void outputAddedElement(char anElement) : 삽입된 원소를 출력한다.
- public void outputStackElement(char anElement) : Stack 원소의 원소를 출력한다.
- public void outputTopElement(char anElement) : Top 의 원소를 출력한다.
- public void outputStackSize(int aStackSize) : 스택의 원소의 개수를 출력한다.
- public void outputRemove(char anElement) : 삭제된 원소를 출력한다.

- `public void outputResult(int numberOfInputChars, int numberOfIgnoredChars, int numberOfAddedChars)` : 입력/추가/무시 된 문자들을 모두 출력하고 종료메시지를 내보낸다.
- `public void outputMessage(String aMessageString)` : 문자열을 출력한다.

■ ArrayList

- `public ArrayList()` : top 초기화, capacity 를 기본값인 5 로 초기화, capacity 만큼 배열 할당
- `public ArrayList(int givenCapacity)` : top 초기화, capacity 를 givenCapacity 로 초기화, capacity 만큼 배열 할당
- `public boolean isEmpty()` : 비었는 지 여부 반환
- `public boolean isFull()` : 다 찼는지 여부 반환
- `public int size()` : 현재 스택의 크기 출력
- `public boolean push(T anElement)` : 스택에 원소 추가
- `public T pop()` : 스택에서 원소 제거 후 반환
- `public T peek()` : 스택의 Top 위치의 원소 반환
- `public void clear()` : 배열의 원소 전부 null 로 초기화
- `public T elementAt(int anOrder)` : 인자로 받은 anOrder 의 위치가 유효할 시 해당 위치의 원소 반환, 유효하지 않을 시 null 반환

c. 종합 설명서

A 부터 Z, a 부터 z 까지의 알파벳을 입력받으면 스택에 Push 한다.

0 부터 9 까지의 숫자를 넣으면, 해당 숫자만큼 스택에서 Pop 한다.

- 를 넣으면, 스택에서 하나를 Pop 한다.

를 넣으면, 현재 스택의 크기를 출력한다.

/ 를 넣으면, 스택의 원소를 Bottom 부터 시작하여 Top 까지 출력한다.

\ 를 넣으면, 스택의 원소를 Top 부터 시작하여 Bottom 까지 출력한다.

^ 를 넣으면, Top 에 위치한 원소를 출력한다.

! 를 넣으면, 스택 원소를 전부 제거하고 입력에 대한 처리 여부를 출력 후 종료 메시지를 출력한다.

이외의 입력이 들어올 시, 오류를 출력하고 무시한다.

2. 프로그램 장단점 분석

장점 : Stack 의 특징을 구현하는 함수가 인터페이스에 정의되어 있기 때문에, Model 의 구조를 파악하기가 용이하다.

3. 실행 결과 분석

a. 입력과 출력

> 프로그램을 시작합니다.
[스택 사용을 시작합니다.]
- 문자를 입력하십시오 : **A**
[Push] 삽입된 원소는 'A' 입니다.
- 문자를 입력하십시오 : **x**
[Push] 삽입된 원소는 'x' 입니다.
- 문자를 입력하십시오 : **#**
[Size] 스택에는 현재 2 개의 원소가 있습니다.
- 문자를 입력하십시오 : **h**
[Push] 삽입된 원소는 'h' 입니다.
- 문자를 입력하십시오 : **/**
[Stack] <Bottom> A x h <Top>
- 문자를 입력하십시오 : **W**
[Push] 삽입된 원소는 'W' 입니다.
- 문자를 입력하십시오 : **z**
[Push] 삽입된 원소는 'z' 입니다.
- 문자를 입력하십시오 : **p**
[Full] 스택이 꽉 차서 삽입이 불가능합니다.
- 문자를 입력하십시오 : ****
[Stack] <Top> z W h x A <Bottom>
- 문자를 입력하십시오 : **-**
[Pop] 삭제된 원소는 'z' 입니다.
- 문자를 입력하십시오 : **^**
[Top] Top 원소는 'W' 입니다.
- 문자를 입력하십시오 : **5**
[Pop] 삭제된 원소는 'W' 입니다.
[Pop] 삭제된 원소는 'h' 입니다.
[Pop] 삭제된 원소는 'x' 입니다.
[Pop] 삭제된 원소는 'A' 입니다.
[Empty] 스택에 삭제할 원소가 없습니다.
- 문자를 입력하십시오 : **B**
[Push] 삽입된 원소는 'B' 입니다.
- 문자를 입력하십시오 : **e**
[Push] 삽입된 원소는 'e' 입니다.
- 문자를 입력하십시오 : **!**
[스택 입력을 종료합니다.]
[Pop] 삭제된 원소는 'e' 입니다.
[Pop] 삭제된 원소는 'B' 입니다.
입력된 문자는 모두 14 개 입니다.
추가된 문자는 모두 7 개 입니다.
무시된 문자는 모두 2 개 입니다.
> 프로그램을 종료합니다.

b. 결과분석

A, x 를 넣고 크기를 확인하니 2 가 나왔다.

h 를 넣고 Bottom 부터 출력하니 나중에 넣은 것일 수록 Top 에 가깝게 잘 들어간 것이 확인되었다.

W, z 를 넣고 p 를 더 넣으려 하자 정해놓은 용량인 5 개가 꽉 차서 들어가지 않았다.

Top 부터 출력하니, 처음에 넣은 것일수록 Bottom 에 가깝게 들어간 것이 확인되었다.

z 를 제거 후 Top 출력을 하였더니 Top 원소인 w 가 잘 출력되었다.

5 개를 제거하려고 하자, 현재 스택에 들어있는 4 개가 제거된 후 비게 되어 오류를 출력하였다.

빈 스택에 B, e 를 순서대로 넣고 종료하니 나중에 넣은 것 부터 Pop 된 후에 종료 메시지를 출력하였다.

입력 : A, x, #, h, /, W, z, p, /, -, ^, 5, B, e -> 14

정상 처리 (추가) : A, x, h, W, z, B, e -> 7

무시 : p, 제거할 때 비어서 안된 것

따라서 알맞는 결과가 나왔다고 할 수 있다.

4. 소스코드

AppController.java

```
public class AppController {

    private AppView _appView;
    private ArrayList<Character> _arrayStack;
    private int _inputChars;           // 입력된 문자의 개수
    private int _ignoredChars; // 무시된 문자의 개수
    private int _addedChars;           // 삽입된 문자의 개수

    // 생성자
    public AppController() {
        this._appView = new AppView();
        this._inputChars = 0;
        this._ignoredChars = 0;
        this._addedChars = 0;
    }

    // 비공개 함수

    // 출력 관련
    // Bottom 부터 전체 출력
```

```

private void showAllFromBottom() {
    this.showMessage(MessageID.Notice_ShowStack);
    this.showMessage(MessageID.Show_StartBottom);

    for(int index = 0; index < this._arrayStack.size(); index++) {
        this._appView.outputStackElement
            ((Character)this._arrayStack.elementAt(index).charValue());
    }
    this.showMessage(MessageID.Show_EndTop);
}

```

// Top 부터 전체 출력

```

private void showAllFromTop() {
    this.showMessage(MessageID.Notice_ShowStack);
    this.showMessage(MessageID.Show_StartTop);

    for(int index = this._arrayStack.size() - 1; index >= 0; index--) {

this._appView.outputStackElement((Character)(this._arrayStack.elementAt(index)).charValue());
    }
    this.showMessage(MessageID.Show_EndBottom);
}

```

// 맨 위 원소 출력

```

private void showTopElement() {
    if(this._arrayStack.isEmpty()) {
        this._appView.outputMessage("[Empty] 스택에 원소가 존재하지 않습니다. \n");
    }
    else {
        Character topElement = this._arrayStack.peek();
        this._appView.outputTopElement(topElement.charValue());
    }
}

```

// 스택 크기 출력

```

private void showStackSize() {
    this._appView.outputStackSize(this._arrayStack.size());
}

```

// 상황에 맞는 메시지 출력

```

private void showMessage(MessageID aMessageID) {
    switch(aMessageID) {

        case Notice_StartProgram :
            this._appView.outputMessage("> 프로그램을 시작합니다. \n");
            break;

        case Notice_EndProgram :
            this._appView.outputMessage("> 프로그램을 종료합니다. \n");
            break;

        case Notice_StartMenu :
            this._appView.outputMessage("[ 스택 사용을 시작합니다. ] \n");
            break;

        case Notice_EndMenu :
            this._appView.outputMessage("[ 스택 입력을 종료합니다. ] \n");

```

```

        break;

    case Notice_InputStack :
        this._appView.outputMessage("[Push] 스택에 원소를 삽입합니다. \n");
        break;

    case Notice_DelStack :
        this._appView.outputMessage("[Pop] 스택의 원소를 제거합니다. \n");
        break;

    case Notice_ShowStack :
        this._appView.outputMessage("[Stack] ");
        break;

    case Show_StartBottom :
        this._appView.outputMessage("<Bottom> ");
        break;

    case Show_StartTop :
        this._appView.outputMessage("<Top> ");
        break;

    case Show_EndTop :
        this._appView.outputMessage(" <Top> \n");
        break;

    case Show_EndBottom :
        this._appView.outputMessage(" <Bottom> \n");
        break;

    case Error_WrongMenu :
        this._appView.outputMessage("[Error] 의미 없는 문자가 입력되었습니다. \n");
        break;

    case Error_InputFull :
        this._appView.outputMessage("[Full] 스택이 꽉 차서 삽입이 불가능합니다. \n");
        break;

    case Error_RemoveEmpty :
        this._appView.outputMessage("[Empty] 스택에 삭제할 원소가 없습니다. \n");
        break;

    }
}

// 횟수 계산 관련
// 추가 / 무시 / 입력된 원소 수 셈
private void countAdded() {
    this._addedChars++;
}
private void countIgnored() {
    this._ignoredChars++;
}
private void countInputChar() {
    this._inputChars++;
}

```

```

// 스택 조작 관련
// 추가
private void addToStack(char inputChar) {
    if(this._arrayStack.push(inputChar)) {
        this._appView.outputAddedElement(inputChar);
        this.countAdded();
    }
    else {
        this.showMessage(MessageID.Error_InputFull);
        this.countIgnored();
    }
}

// 1개 제거 후 출력
private void removeOne() {
    if(this._arrayStack.isEmpty()) {
        this.showMessage(MessageID.Error_RemoveEmpty);
        this.countIgnored();
    }
    else {
        this._appView.outputRemove(this._arrayStack.pop().charValue());
    }
}

// 입력된 개수만큼 제거 후 출력
private void removeN(int numberOfCharsToBeRemoved) {
    for(int i = 0; i < numberOfCharsToBeRemoved; i++) {
        this.removeOne();
    }
}

// 전부 제거 후 출력
private void conclusion() {
    int size = this._arrayStack.size();
    for(int i = 0; i < size; i++) {
        this.removeOne();
    }
    this._appView.outputResult(this._inputChars, this._ignoredChars, this._addedChars);
}

// 공개 함수
public void run() {

    this._arrayStack = new ArrayList<Character>();
    char input;           // 메뉴 입력 저장

    this.showMessage(MessageID.Notice_StartProgram);
    this.showMessage(MessageID.Notice_StartMenu);

    input = this._appView.inputCharacter();

    while(input != '!') {

        this.countInputChar();
        if( (input >= 'a' && input <= 'z') ||
            (input >= 'A' && input <= 'Z') ) {
            this.addToStack(input);

```



```

    }
    else if(input >= '0' && input <= '9') {
        this.removeN(input - '0');
    }
    else if(input == '-') {
        this.removeOne();
    }
    else if(input == '#') {
        this.showStackSize();
    }
    else if(input == '/') {
        this.showAllFromBottom();
    }
    else if(input == '\\') {
        this.showAllFromTop();
    }
    else if(input == '^') {
        this.showTopElement();
    }
    else {
        this.showMessage(MessageID.Error_WrongMenu);
        this.countIgnored();
    }

    input = this._appView.inputCharacter();           // 새로운 입력
}

this.showMessage(MessageID.Notice_EndMenu);
this.conclusion();
this.showMessage(MessageID.Notice_EndProgram);

}

}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 공개 함수

    // 입력 관련
    // 정수 입력받아 반환
    public int inputInt() {
        return Integer.parseInt(this._scanner.nextLine());
    }

    // String 입력받아 반환
    public String inputString() {

```

```

        return this._scanner.nextLine();
    }

    // char 입력받아 반환
    public char inputCharacter() {
        char element;
        System.out.print("- 문자를 입력하시오 : ");
        element = this.inputString().charAt(0);
        return element;
    }

    // 출력 관련
    // 추가된 원소 출력
    public void outputAddedElement(char anElement) {
        System.out.printf("[Push] 삽입된 원소는 '%c' 입니다. \n", anElement);
    }

    // Stack 원소 하나 출력
    public void outputStackElement(char anElement) {
        System.out.printf("%3c", anElement);
    }

    // Top 원소 출력
    public void outputTopElement(char anElement) {
        System.out.printf("[Top] Top 원소는 '%c' 입니다. \n", anElement);
    }

    // Stack의 현재 크기 출력
    public void outputStackSize(int aStackSize) {
        System.out.printf("[Size] 스택에는 현재 %d 개의 원소가 있습니다. \n", aStackSize);
    }

    // 제거된 원소 출력
    public void outputRemove(char anElement) {
        System.out.printf("[Pop] 삭제된 원소는 '%c' 입니다. \n", anElement);
    }

    // 결과 출력
    public void outputResult(int numberOfInputChars,
                             int numberOfIgnoredChars,
                             int numberOfAddedChars) {
        System.out.printf("입력된 문자는 모두 %d 개 입니다. \n", numberOfInputChars);
        System.out.printf("추가된 문자는 모두 %d 개 입니다. \n", numberOfAddedChars);
        System.out.printf("무시된 문자는 모두 %d 개 입니다. \n", numberOfIgnoredChars);
    }

    // String 출력
    public void outputMessage(String aMessageString) {
        System.out.print(aMessageString);
    }
}

```

```
public class ArrayList<T> implements Stack<T> {

    private static final int DEFAULT_CAPACITY = 5;
    private int _capacity;
    private int _top;
    private T[] _elements;

    // 생성자
    @SuppressWarnings("unchecked")
    public ArrayList() {
        this._capacity = ArrayList.DEFAULT_CAPACITY;
        this._top = -1;
        this._elements = (T[])new Object[ArrayList.DEFAULT_CAPACITY];
    }

    @SuppressWarnings("unchecked")
    public ArrayList(int givenCapacity) {
        this._capacity = givenCapacity;
        this._top = -1;
        this._elements = (T[])new Object[this._capacity];
    }

    // 공개 함수

    // 비어있는지 확인
    public boolean isEmpty() {
        return (this._top == -1);
    }

    // 꽉 찼는지 확인
    public boolean isFull() {
        return (this.size() == this._capacity);
    }

    // 총 길이 확인 (top의 인덱스 번호를 통함)
    public int size() {
        return (this._top + 1);
    }

    // 원소 추가 (Top)
    public boolean push(T anElement) {
        if(this.isFull()) {
            return false;
        }
        else {
            this._top++;
            this._elements[this._top] = anElement;
            return true;
        }
    }

    // 원소 제거 (Top)
    public T pop() {
        if(this.isEmpty()) {
```

```

        return null;
    }
    else {
        this._top--;
        return this._elements[this._top + 1];    // 삭제된 원소 반환
    }
}

// Top 의 원소 확인
public T peek() {
    if(this.isEmpty()) {
        return null;
    }
    else {
        return this._elements[this._top];
    }
}

// Stack 초기화
public void clear() {
    this._top = -1;
    for(int index = 0; index < this.size(); index++) {
        this._elements[index] = null;
    }
}

// 주어진 순서의 원소 반환
public T elementAt(int anOrder) {
    if(anOrder >= 0 && anOrder < this.size()) {
        return this._elements[anOrder];
    }
    else {
        return null;
    }
}
}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices :
    Notice_StartProgram,
    Notice_EndProgram,
    Notice_StartMenu,
    Notice_EndMenu,
    Notice_InputStack,
    Notice_DeIStack,
    Notice_ShowStack,

    // Message IDs for Shows :
    Show_StartBottom,    // Bottom 에서 시작
    Show_StartTop,       // Top 에서 시작
    Show_EndTop,         // Top 에서 End
    Show_EndBottom,      // Bottom 에서 End
}

```

```
// Message IDs for Errors :
Error_WrongMenu,
Error_InputFull,
Error_RemoveEmpty,

}
```

Stack.java

```
public interface Stack<T> {

    public boolean isEmpty();           // 비어있는지 확인
    public boolean isFull();           // 꽉 찼는지 확인
    public boolean push(T anElement); // 추가
    public T pop();                     // 제거
    public T peek();                   // Top 의 원소 확인

}
```

DS1_07_201702081_최재범.java

```
public class DS1_07_201702081_최재범 {
    public static void main(String[] args) {

        AppController appController = new AppController();
        appController.run();

    }
}
```